SIMATS ENGINEERING

CO2_AT2 Reverse Engineering Task

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A partially tested ATM withdrawal module is provided where only input-output behavior is known (e.g., withdrawal limits, balance validation, error messages). Reverse engineer the system by analyzing the observed behavior and reconstructing the underlying test case design strategy. Decompose the system into functional components and identify possible equivalence classes and boundary conditions. Infer the black box testing techniques used and evaluate their adequacy. Reconstruct an improved set of test cases covering all critical scenarios. Document your analysis with structured explanation.	5
2	A software module's execution traces and code snippets are given, but no test documentation is available. Reverse engineer the white box testing approach by constructing the control flow graph from the given logic. Identify all decision points, loops, and independent execution paths. Evaluate the existing test adequacy in terms of coverage criteria. Reconstruct a complete path testing strategy ensuring full coverage. Present your findings with clear reasoning and structured representation.	5
3	A web application login and transaction system has been tested using unknown techniques, but only defect logs and test results are available. Reverse engineer the testing approach by analyzing defect patterns and identifying which black box and white box techniques were likely used. Decompose the system into testable components and evaluate gaps in testing coverage. Reconstruct a comprehensive testing strategy integrating equivalence partitioning, boundary value analysis, and path testing. Justify your reconstructed approach with logical reasoning and structured documentation.	5

4	A legacy inventory management system provides only a set of successful and failed transaction logs regarding stock updates and low-stock alerts. Reverse engineer the business logic by analyzing these logs to identify the hidden boundary conditions and equivalence partitions. Decompose the update process into functional units and reconstruct a black box test design that addresses the gaps found in the logs. Justify your selection of test cases based on the inferred logic.	5
5	You are provided with a compiled .exe file of a library management module and its corresponding CPU/Memory execution profile during a stress test. Reverse engineer the white box testing requirements by mapping execution spikes to potential loops and decision clusters in the logic. Construct a conceptual control flow graph (CFG) that represents this behavior. Reconstruct a path testing strategy that targets these resource-intensive paths and evaluate its impact on system stability.	6
6	A mobile payment gateway's API responses (JSON) for various transaction states—Pending, Success, Reversal, and Timeout—are given. Reverse engineer the underlying state machine by analyzing the transition triggers and data constraints. Identify the implicit state transition testing technique used. Reconstruct a comprehensive state-based test suite that ensures all legal and illegal transitions are covered. Document your analysis with a structured state-transition diagram.	5
7	An encrypted communication module provides only "black-box" behavior through its input parameters and encrypted outputs. Reverse engineer the validation logic by performing "fuzzing" analysis on the input fields to identify error-handling boundaries. Decompose the module into data validation and encryption components. Reconstruct an improved set of test cases using Boundary Value Analysis (BVA) and justify how your design prevents buffer overflow or injection vulnerabilities.	5
8	A set of legacy unit test results for an insurance premium calculator shows 100% statement coverage but fails to catch defects in multi-condition "if" statements. Reverse engineer the original white box approach and identify the inadequacy in the coverage criteria used. Construct a new CFG and identify all independent execution paths. Reconstruct a path testing strategy that achieves 100% Branch and Path coverage. Explain how this reconstruction reduces the risk of calculation errors.	5

9	A cloud-based file synchronization tool provides user activity logs and a list of "lost file" defect reports. Reverse engineer the synchronization algorithm's logic by analyzing the defect patterns related to file size, type, and network speed. Infer the likely Equivalence Class Partitioning (ECP) used by the original developers. Reconstruct a revised testing strategy that integrates BVA with stress testing. Justify how your reconstructed approach prevents future data loss.	5
10	You are given a sequence of database queries (SQL) generated by an e-commerce "Search and Filter" module, but no source code. Reverse engineer the search logic by analyzing the query parameters and the resulting dataset. Identify the decision-making logic used for complex filtering (e.g., AND/OR/NOT). Reconstruct a Decision Table based testing strategy that covers all possible search combinations. Evaluate the completeness of your reconstructed model.	5

ANSWERS:

QUESTION 1. ATM Withdrawal Module – Reverse Engineering Black-Box Testing

1. Understanding the System

An ATM withdrawal module validates the withdrawal amount, account balance, and transaction limits before dispensing cash. Since only input-output behavior is available, **black-box reverse engineering** is used to infer the underlying business rules and reconstruct the test cases.

For this analysis, assume that the minimum withdrawal is ₹100, the maximum withdrawal per transaction is ₹20,000, and the withdrawal amount must be a multiple of ₹100.

2. Functional Decomposition

Component	Function	Possible Defect
Input Handler	Accepts withdrawal amount	Invalid input accepted
Amount Validator	Checks minimum, maximum, and denomination	Incorrect boundary validation
Balance Validator	Checks available balance	Insufficient-fund withdrawal
Limit Validator	Checks daily withdrawal limit	Limit exceeded
Transaction Processor	Deducts money from the account	Incorrect balance update
Cash Dispenser	Dispenses cash	Incorrect cash amount
Error Handler	Displays error messages	Incorrect error message

3. Equivalence Class Partitioning (ECP)

ECP divides the input values into valid and invalid classes.

Equivalence Class	Example Input	Expected Result
Negative amount	-₹100	Reject
Zero amount	₹0	Reject
Below minimum	₹50	Reject
Valid amount	₹1,000	Accept if other checks pass
Above maximum	₹20,100	Reject
Invalid denomination	₹150	Reject
Insufficient balance	₹5,000 when balance is ₹2,000	Reject
Daily limit exceeded	Amount exceeds remaining daily limit	Reject

4. Boundary Value Analysis (BVA)

Test Case	Withdrawal Amount	Expected Result
TC1	₹99	Reject
TC2	₹100	Accept if other checks pass
TC3	₹101	Reject if ₹100 increments are mandatory
TC4	₹19,999	Reject if ₹100 increments are mandatory
TC5	₹20,000	Accept if other checks pass
TC6	₹20,001	Reject

Note: The minimum and maximum values are valid, but amounts immediately above or below the permitted limits must be rejected. Denomination validation is a separate rule from amount-range validation.

5. Reconstructed Testing Strategy

The improved test suite should:

1. Apply ECP to valid and invalid withdrawal amounts.
2. Apply BVA around the minimum and maximum limits.
3. Test insufficient balance and daily withdrawal limits.
4. Verify that failed withdrawals do not change the account balance.
5. Verify that successful withdrawals deduct and dispense the correct amount exactly once.
6. Test network failures, duplicate requests, and incorrect PINs.
7. Check whether the appropriate error message is displayed for each failure.

Conclusion

ECP reduces redundant tests, while BVA identifies defects near critical withdrawal limits. Combining these techniques with balance validation and transaction-integrity testing improves defect detection, financial accuracy, and ATM reliability.

QUESTION 2. Software Module – Reverse Engineering White-Box Testing

1. Understanding the System

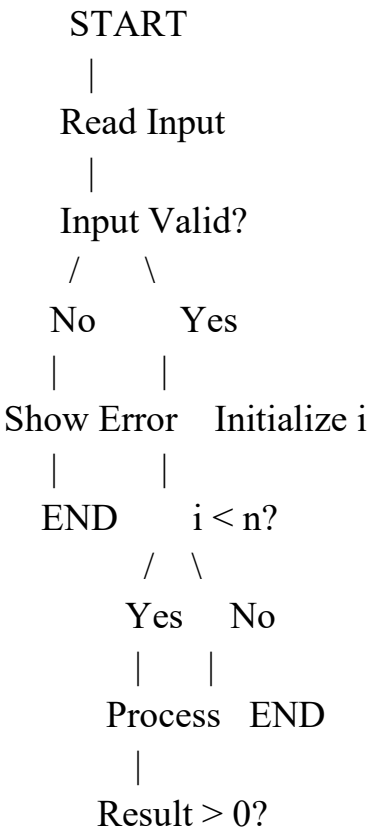
The module provides source-code snippets and execution traces but no testing documentation. The objective is to reconstruct the control flow, identify decision points and loops, and determine whether the existing test suite provides adequate coverage.

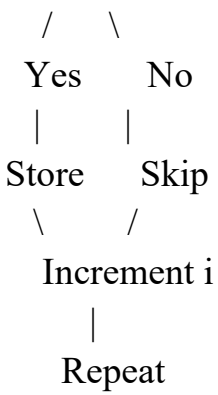
2. Functional Decomposition

Component	Function
Input Module	Reads and validates input
Decision Module	Evaluates conditions
Processing Module	Executes the required operation
Loop Controller	Controls repeated execution
Error Handler	Handles invalid input
Output Module	Displays the result

3. Conceptual Control Flow Graph (CFG)

Consider the following illustrative logic:





The graph contains input validation, a nested decision, and a loop.

4. Independent Execution Paths

The following principal paths can be identified:

- **Path 1:** Invalid input → error message → termination.
- **Path 2:** Valid input → loop condition false → termination.
- **Path 3:** Valid input → loop entered → result positive → store result.
- **Path 4:** Valid input → loop entered → result non-positive → skip storage.

These paths are illustrative; the actual paths must be derived from the supplied code and execution traces.

5. Coverage Analysis

- **Statement coverage:** Every executable statement is executed.
- **Branch coverage:** Every decision outcome is executed.
- **Path coverage:** Relevant execution paths are tested.
- **Loop testing:** Zero, one, two, and multiple iterations are tested.

Statement coverage alone cannot guarantee branch coverage. Similarly, complete path coverage may be impractical when loops create many possible execution paths.

Cyclomatic complexity is calculated as:

$$V(G) = E - N + 2P$$

where E is the number of edges, N is the number of nodes, and P is the number of connected components. For a single connected graph:

$$V(G) = E - N + 2$$

6. Reconstructed Testing Strategy

1. Construct the CFG from the actual source code.
2. Identify all decisions, loops, and feasible paths.
3. Calculate cyclomatic complexity.
4. Design tests for every branch outcome.
5. Test loops with zero, one, two, and multiple iterations.
6. Compare execution traces with expected paths.
7. Measure statement and branch coverage.
8. Add regression tests for identified defects.

Conclusion

CFG construction and basis path testing provide a systematic way to identify missing execution paths. The reconstructed strategy improves structural coverage and reduces defects caused by untested conditions and loop behavior.

3. Web Application Login and Transaction System – Hybrid Reverse Engineering

1. Understanding the System

The web application includes login, authentication, session management, and transaction processing. Only defect logs and test results are available, so the original testing techniques must be inferred from the observed failure patterns.

For example, failures at password-length limits may indicate inadequate boundary testing, while transaction failures under specific conditions may indicate missing decision or path coverage.

These are hypotheses that should be verified against the actual defect logs.

2. Functional Decomposition

Component	Function	Testing Focus
Login Interface	Accepts credentials	Empty and malformed inputs
Authentication Module	Verifies credentials	Valid and invalid passwords
Session Manager	Maintains sessions	Expiration and logout
Authorization Module	Checks permissions	Unauthorized access
Transaction Validator	Validates transaction details	Amount and account limits
Transaction Processor	Executes transactions	Correct balance updates
Error Handler	Reports failures	Error messages and recovery

3. Possible Gaps in Existing Testing

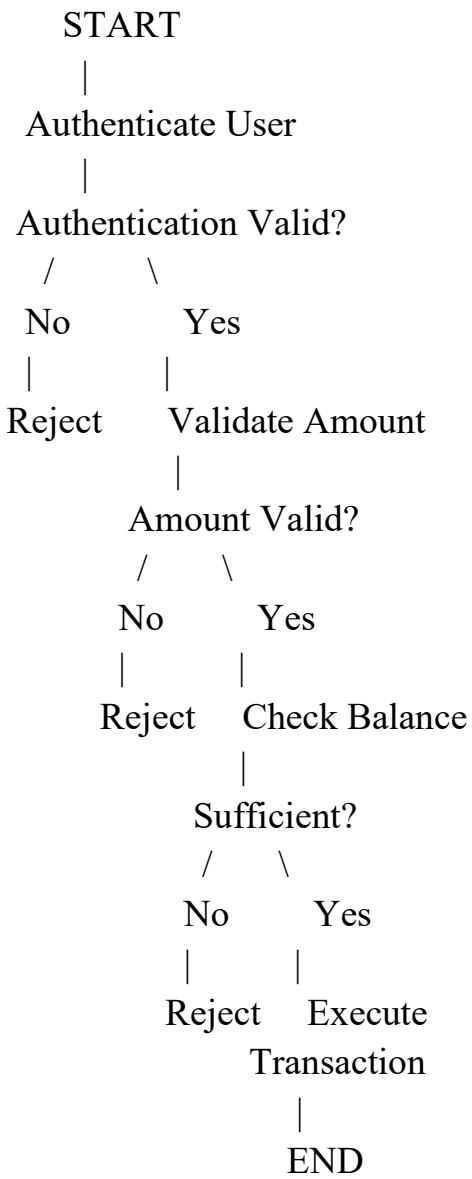
Defect Pattern	Possible Missing Technique
Errors at input limits	Boundary Value Analysis
Errors in particular input categories	Equivalence Class Partitioning
Errors involving combinations of conditions	Decision Table Testing
Untested decision outcomes	Branch Coverage
Errors after particular action sequences	State Transition Testing

4. Reconstructed Black-Box Test Cases

Assume the permitted transaction amount is ₹100 to ₹50,000.

Test Case	Input	Expected Result
TC1	Valid username and password	Login succeeds
TC2	Incorrect password	Login rejected
TC3	Empty username	Validation error
TC4	Transaction amount ₹99	Reject
TC5	Transaction amount ₹100	Accept if other checks pass
TC6	Transaction amount ₹50,001	Reject
TC7	Insufficient account balance	Reject
TC8	Duplicate transaction request	Prevent duplicate debit

5. White-Box Path Testing



The principal paths are:

1. Authentication failure.
2. Successful authentication followed by invalid transaction amount.
3. Successful authentication, valid amount, insufficient balance.
4. Successful authentication, valid amount, sufficient balance.

6. Improved Hybrid Strategy

- Apply ECP to credentials and transaction inputs.
- Apply BVA to password-length and transaction-amount boundaries.
- Use decision tables for combinations of transaction conditions.
- Construct CFGs for authentication and transaction processing.
- Test all feasible branches and independent paths.
- Test session expiration and authorization failures.
- Verify transaction integrity and duplicate-request handling.
- Perform security and regression testing.

Conclusion

Hybrid reverse engineering combines evidence from defect logs with black-box and white-box testing techniques. The reconstructed strategy improves input coverage, control-flow coverage, security, and transaction reliability.

4. Inventory Management System – Reverse Engineering Business Logic

1. Understanding the System

The inventory system provides successful and failed transaction logs involving stock updates and low-stock alerts. By comparing input quantities, resulting stock levels, and alert outputs, the tester can infer the hidden business rules.

For illustration, assume that negative stock is prohibited and a low-stock alert is generated whenever stock falls below 10 units.

2. Functional Decomposition

Component	Function
Input Handler	Accepts stock-update requests
Quantity Validator	Checks quantity values
Stock Update Module	Adds or removes stock
Threshold Checker	Compares stock against the reorder threshold
Alert Module	Generates low-stock notifications
Database Module	Stores inventory records
Error Handler	Rejects invalid transactions

3. Inferring Business Rules from Logs

Suppose the observed logs contain the following results:

- Stock 15 − 5 = 10 → no alert.
- Stock 15 − 6 = 9 → alert generated.
- Stock 5 + 3 = 8 → alert remains active.

These observations suggest the rule:

$$\text{Low-stock alert} \iff \text{Stock} < 10$$

The rule must be verified against the full requirements and additional logs.

4. Equivalence Class Partitioning

Class	Resulting Stock	Expected Behavior
Negative stock	-1	Reject update
Zero stock	0	Accept only if permitted; trigger alert
Low stock	1–9	Generate alert
Threshold stock	10	No alert under the assumed rule
Normal stock	Above 10	No alert

5. Boundary Value Analysis

Test Case	Resulting Stock	Expected Result
TC1	-1	Reject
TC2	0	Apply zero-stock policy
TC3	1	Alert
TC4	9	Alert
TC5	10	No alert
TC6	11	No alert

These cases test the resulting stock level. Separate cases should validate the quantity entered in each stock-update request.

6. Reconstructed Test Strategy

1. Analyse successful and failed transaction logs.
2. Infer stock-validation and alert conditions.
3. Apply ECP to stock levels and update operations.
4. Apply BVA around zero and the low-stock threshold.
5. Test additions and reductions that cross the threshold.
6. Verify database updates and alert generation.
7. Test simultaneous updates to identify lost-update defects.
8. Confirm that rejected operations do not corrupt inventory records.

Conclusion

Reverse engineering transaction logs helps reconstruct inventory rules and identify missing boundary tests. ECP, BVA, and transaction-integrity testing improve stock accuracy and ensure reliable low-stock alerts.

5. Compiled Library Management Module – CPU/Memory Analysis and Path Testing

1. Understanding the System

A compiled .exe file and its CPU/memory execution profile are available, but the source code is missing. The goal is to identify resource-intensive execution scenarios and reconstruct appropriate white-box testing requirements.

CPU and memory spikes can indicate inefficient loops, repeated operations, or memory-management problems. However, profiles alone cannot prove the exact internal control flow.

2. Functional Decomposition

A typical library management module may contain:

Component	Function
Search Module	Finds books and records
Borrowing Module	Processes book loans
Return Module	Updates returned-book records
Fine Calculator	Calculates overdue charges
Database Module	Reads and writes records
Report Generator	Produces summaries
Memory Manager	Allocates and releases memory

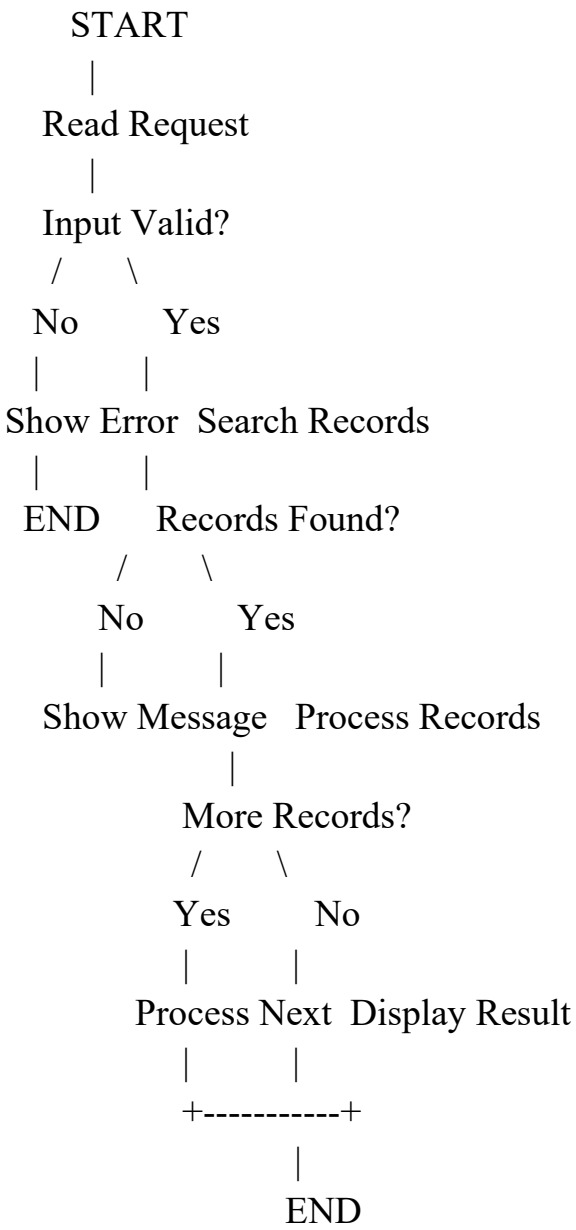
These components must be verified against the executable's observable behavior.

3. Interpreting Resource Profiles

Observation	Possible Cause	Testing Action
High CPU during searches	Repeated scanning	Increase the number of records
Memory spikes during reports	Large result sets	Generate reports with increasing dataset sizes
CPU spikes during borrowing	Repeated validation	Test large transaction histories
Memory grows after repeated operations	Possible memory leak	Repeat operations and monitor memory
Long delays on large datasets	Inefficient processing	Test different dataset sizes

4. Conceptual Control Flow Graph

The following illustrative CFG represents a search-and-processing operation:



This is a conceptual model, not a verified representation of the compiled executable's actual control flow.

5. Reconstructed Path Testing Strategy

1. Establish baseline CPU and memory measurements.
2. Test empty, small, medium, and large datasets.
3. Test successful and unsuccessful searches.
4. Test large borrowing histories and report generation.
5. Repeat operations to detect memory growth.
6. Measure resource consumption under different workloads.
7. Use authorized profiling, debugging, or instrumentation tools if available.
8. If source-level access becomes available, construct the actual CFG, calculate cyclomatic complexity, and test independent paths.

6. Expected Impact

This approach identifies resource-intensive workloads and helps distinguish computational bottlenecks from memory-management problems. Repeated measurements also show whether optimizations improve performance and stability.

Conclusion

Performance profiles support reverse engineering by identifying scenarios that require deeper investigation. Combining workload testing with profiling or source-level instrumentation provides stronger evidence for reconstructing control flow and improving application stability.

6. Mobile Payment Gateway – Reverse Engineering State Transitions

1. Understanding the System

The mobile payment gateway returns JSON responses for Pending, Success, Reversal, and Timeout. The objective is to infer the payment state machine and identify legal and illegal transitions.

Payment processing requires careful state management because duplicate requests, delayed responses, and repeated reversals can cause financial inconsistencies.

2. Functional Decomposition

Component	Function
API Handler	Receives payment requests
Payment Validator	Validates amount and transaction details
Payment Processor	Initiates payment
State Manager	Tracks transaction status
Timeout Handler	Handles delayed responses
Reversal Handler	Processes reversals

Response Generator Returns JSON responses

3. Reconstructed State-Transition Diagram

Assume that Timeout is a recoverable state and Reversal is permitted only after a successful payment.

The actual API specification must determine whether a timed-out transaction can later succeed and which terminal states permit reversals.

4. State-Transition Test Cases

Test Case	Current State	Event	Expected Result
TC1	New transaction	Initiate payment	Pending
TC2	Pending	Payment approved	Success
TC3	Pending	Response deadline exceeded	Timeout

Test Case	Current State	Event	Expected Result
TC4	Success	Valid reversal request	Reversal
TC5	Timeout	Late success response	Apply documented reconciliation policy
TC6	Reversal	Duplicate reversal request	Reject or return existing status
TC7	Success	Duplicate success notification	No duplicate debit
TC8	Reversal	New reversal request	Reject or return existing reversal status

5. Improved State-Based Testing Strategy

1. Identify all states and transition triggers from API responses.
2. Document the required JSON fields and status constraints.
3. Test every legal transition.
4. Attempt illegal transitions and verify correct rejection.
5. Test delayed responses, duplicate notifications, and retries.
6. Verify transaction IDs, amounts, and state consistency.
7. Test idempotency to prevent duplicate debits or reversals.
8. Perform regression testing after correcting state-management defects.

Conclusion

State-transition testing is more suitable than testing individual input values alone because payment correctness depends on the sequence of events. Reconstructing the state machine improves transaction integrity and prevents invalid transitions and duplicate processing.

7. Encrypted Communication Module – Fuzzing and Boundary Value Analysis

1. Understanding the System

The encrypted communication module accepts input parameters and returns encrypted outputs, but its internal implementation is unavailable. The tester must infer input-validation rules by supplying valid, invalid, and boundary-value inputs.

Fuzzing helps expose unexpected failures, while BVA systematically tests known input limits.

2. Functional Decomposition

Component	Function
Input Parser	Reads incoming parameters
Data Validator	Checks length, type, and format
Buffer Manager	Manages input and output memory
Encryption Engine	Encrypts valid data

Component	Function
Output Encoder	Formats the encrypted result
Error Handler	Handles invalid input safely

3. Fuzzing Test Inputs

The following inputs should be tested in an authorized, isolated environment:

- Empty input.
- Missing parameters.
- Invalid data types.
- Very long strings.
- Malformed encodings.
- Unexpected Unicode characters.
- Embedded control characters.
- Oversized payloads.
- Repeated delimiters and malformed structured input.

Observe whether the system returns a valid result, rejects the input safely, crashes, hangs, or exposes sensitive information.

4. Boundary Value Analysis

Assume the documented plaintext input length is 1–4,096 bytes. This is an illustrative specification.

Test Case	Input Length	Expected Result
TC1	0 bytes	Reject if empty input is prohibited
TC2	1 byte	Accept
TC3	2 bytes	Accept
TC4	4,095 bytes	Accept
TC5	4,096 bytes	Accept
TC6	4,097 bytes	Reject safely

Additional tests should cover encoding restrictions and any block-size constraints defined by the encryption algorithm.

5. Reconstructed Testing Strategy

1. Identify input fields and their documented constraints.
2. Apply ECP to valid and invalid data types.
3. Apply BVA to input-length boundaries.
4. Perform fuzz testing on malformed and unexpected inputs.
5. Verify that invalid inputs do not crash or hang the module.
6. Use approved tools to examine memory safety and error handling.
7. Test encryption and decryption consistency for valid inputs.
8. Verify that validation cannot be bypassed.

9. Repeat tests after fixing identified defects.

Conclusion

BVA provides systematic boundary testing, while fuzzing explores unexpected inputs. Combining both techniques helps identify validation weaknesses and reduces the risk of memory-safety and injection vulnerabilities. Additional security analysis is required to establish that the module is safe.

8. Insurance Premium Calculator – Reverse Engineering Coverage Gaps

1. Understanding the System

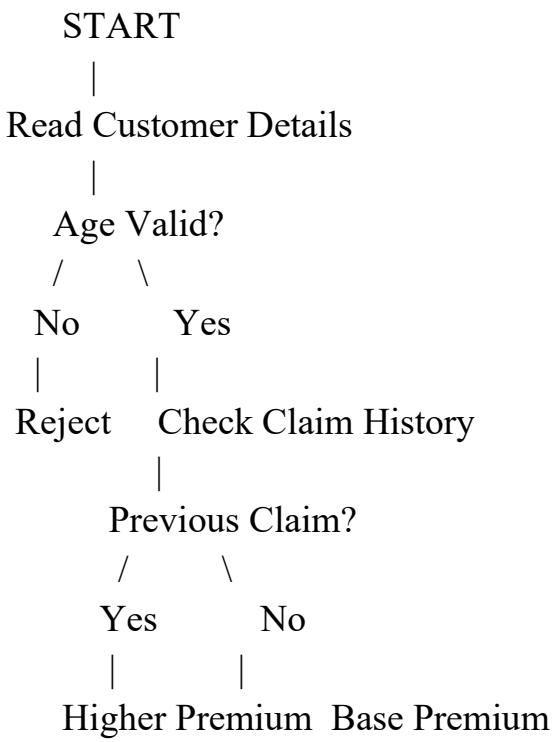
The insurance premium calculator has 100% statement coverage, but defects remain in multi-condition if statements. This suggests that the existing tests execute the statements without adequately testing the decision outcomes and combinations of Boolean conditions.

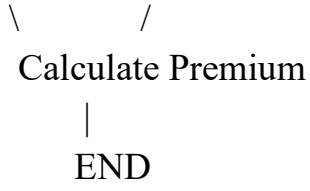
2. Functional Decomposition

Component	Function
Input Validator	Validates customer details
Age Classifier	Determines age group
Claim Evaluator	Checks claim history
Policy Classifier	Identifies policy type
Premium Calculator	Calculates the premium
Output Module	Displays the final amount

3. Conceptual Control Flow Graph

Consider this illustrative logic:





4. Independent Execution Paths

1. Invalid age → reject input.
2. Valid age → previous claim → apply higher premium.
3. Valid age → no previous claim → apply base premium.

The actual CFG must include every decision in the real implementation.

5. Why Statement Coverage Was Insufficient

Statement coverage verifies that every executable statement runs at least once. It does not prove that each decision outcome has been tested.

For example, a test may execute the higher-premium statement without checking every relevant combination of age, claim history, and policy type.

- **Branch coverage:** Tests every decision outcome.
- **Condition coverage:** Makes each Boolean condition evaluate to both True and False.
- **Path testing:** Tests independent feasible execution paths.
- **Modified Condition/Decision Coverage (MC/DC):** Demonstrates that each condition can independently affect the overall decision outcome.

6. Reconstructed Testing Strategy

1. Construct the CFG from the actual source code.
2. Identify all Boolean conditions and decision points.
3. Calculate cyclomatic complexity.
4. Test each feasible independent path.
5. Achieve 100% branch coverage for the defined module.
6. Apply MC/DC when independent condition effects must be demonstrated.
7. Verify premium calculations using independently calculated expected values.
8. Test boundary ages and invalid customer data.
9. Perform regression testing after correcting calculation errors.

Conclusion

The original testing approach relied too heavily on statement coverage. CFG analysis, branch testing, and independent path testing expose missing decision outcomes and improve the accuracy and reliability of premium calculations.

Note: A basis-path test suite does not automatically guarantee 100% path coverage. The achieved coverage must be measured against the actual program structure.

9. Cloud File Synchronization Tool – Reverse Engineering ECP and BVA

1. Understanding the System

The cloud file synchronization tool has reported lost-file defects associated with file size, file type, and network speed. These failures may result from inadequate input partitions, interrupted transfers, synchronization conflicts, or resource limitations.

2. Functional Decomposition

Component	Function
File Scanner	Detects file changes
File Validator	Checks type and size
Upload Manager	Uploads local files
Download Manager	Downloads remote files
Synchronization Engine	Coordinates file updates
Conflict Resolver	Handles simultaneous modifications
Network Handler	Manages interruptions and retries
Storage Manager	Maintains file integrity and metadata

3. Inferring the Original Equivalence Classes

File-size classes:

- Empty files.
- Small files.
- Normal files.
- Large files.
- Files exceeding the supported limit.

File-type classes:

- Supported formats.
- Unsupported formats.
- Files with missing or unusual extensions.

Network-condition classes:

- Stable, high-speed connection.
- Slow connection.
- Intermittent connection.
- Complete connection loss.

If defect reports cluster around large files or slow connections, the original test suite may have underrepresented these conditions.

4. Boundary Value Analysis

Assume the maximum supported file size is 100 MB.

Test Case	File Size	Expected Result
TC1	0 bytes	Follow empty-file policy
TC2	1 byte	Synchronize if supported
TC3	99 MB	Synchronize successfully
TC4	100 MB	Synchronize successfully
TC5	101 MB	Reject or handle according to specification

5. Improved Testing Strategy

1. Apply ECP to file sizes, file types, and network conditions.
2. Apply BVA to file-size limits.
3. Test uploads and downloads under slow and interrupted connections.
4. Perform stress tests with many files and large datasets.
5. Verify file hashes before and after synchronization.
6. Test simultaneous modifications and conflict resolution.
7. Verify retry and recovery after network failures.
8. Confirm that partial transfers are not reported as complete.
9. Perform regression testing after fixing synchronization defects.

6. Justification

ECP reduces redundant tests by selecting representative input classes. BVA detects errors near supported file-size limits. Stress testing identifies failures under heavy workloads, while hash verification directly checks file integrity.

Conclusion

Combining ECP, BVA, stress testing, and integrity verification improves synchronization reliability. The reconstructed strategy targets the conditions associated with lost-file defects and reduces the risk of future data loss.

10. E-Commerce Search and Filter – Reverse Engineering Decision Logic

1. Understanding the System

The e-commerce search module generates SQL queries, but the source code is unavailable. By analysing query parameters and returned datasets, the tester can infer how filters are combined using AND, OR, and NOT.

Incorrect logical operators or missing conditions may return unwanted products or exclude valid products.

2. Functional Decomposition

Component	Function
Search Interface	Accepts search terms
Parameter Validator	Validates filter inputs

Component	Function
Query Builder	Constructs SQL conditions
Database Engine	Executes queries
Result Processor	Formats matching products
Pagination Module	Handles result pages
Security Layer	Protects against SQL injection

3. Inferring the Search Logic

Consider this illustrative SQL query:

```
SELECT *
FROM Products
WHERE category = 'Electronics'
  AND price BETWEEN 1000 AND 5000
  AND (brand = 'A' OR brand = 'B')
  AND NOT discontinued = TRUE;
```

The query requires the product to:

1. Belong to the Electronics category.
2. Have a price between ₹1,000 and ₹5,000, inclusive.
3. Belong to brand A or brand B.
4. Not be discontinued.

This example illustrates how query behavior can be reverse-engineered; it is not the actual query supplied by the question.

4. Decision Table Testing

Let:

- **C1:** Category matches.
- **C2:** Price is within range.
- **C3:** Brand is A or B.
- **C4:** Product is not discontinued.

Test Case	C1	C2	C3	C4	Expected Result
TC1	True	True	True	True	Include product
TC2	False	True	True	True	Exclude product
TC3	True	False	True	True	Exclude product
TC4	True	True	False	True	Exclude product
TC5	True	True	True	False	Exclude product
TC6	True	True	Brand A	True	Include product
TC7	True	True	Brand B	True	Include product

TC6 and TC7 explicitly test both alternatives of the OR condition.

5. Reconstructed Testing Strategy

1. Analyse the generated SQL queries and returned datasets.
2. Identify each filtering condition and logical operator.
3. Construct decision tables for meaningful combinations.
4. Test AND, OR, and NOT behavior.
5. Test minimum and maximum price boundaries.
6. Test missing filters, conflicting filters, and empty results.
7. Compare actual results with independently calculated expected datasets.
8. Test pagination and sorting.
9. Verify the use of parameterized queries and test malformed inputs for SQL injection resistance.

6. Evaluating Completeness

Four independent Boolean conditions can produce up to:

$$2^4 = 16$$

possible combinations.

However, the final test suite must cover all meaningful business rules and both alternatives of the OR condition. Redundant combinations can be reduced only when the reduction is justified and does not leave a requirement untested.

The reconstructed model is considered adequate when its rules have been validated against the actual SQL queries, business requirements, and observed results.

Conclusion

Reverse engineering the SQL queries helps reconstruct the search module's decision logic. Decision table testing systematically covers filter combinations, while result-set comparison and security testing improve correctness and robustness.

Code of Conduct:

I __Haritha[192421282]__ *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:3

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation